

Helix AI: Idea and Problem Statement (In-Depth)

Simple words first, deep engineering details next

Generated: 2026-04-05 01:24:58

This report explains why the project exists, what exact problem it solves, where boundaries are, and how success should be measured.

Project Snapshot	Value
Primary focus	Legacy Python modernization
Architecture mode	Hybrid deterministic + optional ML
Current test status	pass (10 tests)
Backend LOC	1158
Frontend LOC	2647
Test LOC	157
Detected endpoints	10
Migration transform rules	15
Rule benchmark pass	4/4
Unit test status	pass (10 tests)

How to read this document: each section starts with a simple explanation, then a deeper engineering explanation, then a practical checklist.

0. Project Idea in 60 Seconds

Simple Explanation

In simple words: Helix AI takes old Python code, finds outdated syntax, upgrades what is safe, and asks for review where behavior can change.

Deep Explanation

This project is focused and practical: legacy modernization, not full general-purpose refactoring.

The active backend currently exposes 10 API routes to support analyze, modernize, run history, health, and model-status flows.

The architecture keeps deterministic rules as the source of truth and treats ML as an optional assistant.

Practical Checklist

- Say the goal as old-to-modern migration with safety gates.
- Explain that diff-first review is a core product decision.
- Mention that unsupported or risky semantics are flagged, not silently auto-fixed.

Code References

```
main.py
core/execution.py
core/validation.py
```

1. Problem Definition

Simple Explanation

In simple words: old Python code breaks on modern runtimes because keywords, builtins, and APIs changed.

Deep Explanation

Analyzer currently tracks around 15 explicit legacy syntax/API patterns and classifies high-risk semantics separately.

Examples include print statement syntax, xrange/raw_input, dict iterator APIs, has_key, backtick repr, and apply builtin usage.

A migration tool must separate syntax conversion from behavior-sensitive conversion.

Practical Checklist

- Differentiate syntax breakages vs semantic breakages.
- Call out string/bytes and division behavior as high review areas.
- Show that issue detection is explicit and testable.

Code References

```
agents/analyzer.py
```

2. Why Existing Generic Tools Are Not Enough

Simple Explanation

In simple words: general AI coding tools are good at broad coding, but they are not always deterministic for migration safety.

Deep Explanation

Helix AI optimizes for repeatability: same input should produce predictable transforms and transparent risk reports.

The planner and validator enforce a guarded path; this is different from free-form generation-only workflows.

The run history API enables auditability for educational and engineering reviews.

Practical Checklist

- Position Helix as specialized, not universally superior.
- Use deterministic + validation as your key differentiator.
- Always mention traceable diffs and run logs.

Code References

```
core/run_store.py  
core/report_generator.py
```

3. Scope and Boundaries

Simple Explanation

In simple words: this version strongly targets Python 2.x and selected Python 1-era constructs; it does not promise perfect migration for every historical edge case.

Deep Explanation

The project intentionally prioritizes high-frequency legacy constructs seen in real educational or maintenance scenarios.

Unsupported complex migrations are surfaced via warnings and validation failures, which keeps the product honest and safer.

This boundary-first design makes the system deliverable for a student project while still technically credible.

Practical Checklist

- Be explicit about what is supported now.
- Use roadmap language for broader Python-version coverage.
- Avoid claiming 100 percent automatic migration for every file.

Code References

```
README.md  
docs/ARCHITECTURE.md
```

4. Success Criteria

Simple Explanation

In simple words: success means safe upgrades, clear diffs, and no silent breakage.

Deep Explanation

Current benchmark script reports rule pass 4/4 (100.0 percent) on the bundled benchmark set.

Unit test suite currently reports status 'pass' over 10 discovered tests in this environment.

Human review experience is measured by clarity: issue list, planned transforms, diff tab, output tab, and report tab.

Practical Checklist

- Track pass rate, warning rate, and manual-edit rate.
- Track validation failures by category.
- Track turnaround time from paste to reviewed diff.

Code References

```
ml/evaluate_modernizer.py  
tests/test_pipeline.py  
tests/test_api.py
```

5. Hybrid ML + Rule Strategy

Simple Explanation

In simple words: ML can suggest and rank, but rules and validation decide what is accepted.

Deep Explanation

The ML reasoner is optional and environment-driven; core modernization remains available even with ML disabled.

This architecture protects reliability in low-resource or no-GPU environments.

Adapter-based inference enables future improvements without rewriting the main API layer.

Practical Checklist

- Treat ML as a confidence booster, not an uncontrolled executor.
- Keep deterministic fallback always active.
- Expose model status in API and UI for transparency.

Code References

```
core/ml_reasoner.py  
core/config.py  
main.py
```

6. Risk and Governance

Simple Explanation

In simple words: do not auto-apply risky semantic changes without explicit visibility.

Deep Explanation

Analyzer currently flags at least 2 high-risk classes plus medium-risk semantic areas.

Validation blocks output when legacy patterns remain or migrated code becomes syntactically invalid.

A run record stores context for debugging and project defense.

Practical Checklist

- Never hide warnings from the user.
- Fail closed when validation fails.
- Keep run logs for reproducibility.

Code References

```
agents/analyzer.py  
core/validation.py  
core/run_store.py
```

7. Roadmap

Simple Explanation

In simple words: next steps are broader pattern coverage, stronger ML ranking, and larger evaluation sets.

Deep Explanation

Near-term technical upgrades include richer AST-based transforms for complex expressions and function signatures.

Medium-term upgrades include larger benchmark pools and mutation tests for semantic regression detection.

Long-term work includes automated repository-level migration planning with dependency-aware ordering.

Practical Checklist

- Keep roadmap split into near, medium, long term.
- Tie each roadmap item to a measurable KPI.
- Preserve deterministic safety baseline through all upgrades.

Code References

```
docs/VIVA_NOTES.md  
docs/PROJECT_ABSTRACT.md
```

Viva Quick Answers

Q: Why did you choose this problem?

Because legacy Python migration is high-friction, high-risk, and still common in real codebases.

Q: Why not pure ML only?

Pure ML is flexible but less deterministic; we need reproducibility and safety gates for migration.

Q: What is your strongest differentiator?

Deterministic transforms with validation and diff-first review in a single guided workflow.

Q: What is your biggest current limitation?

Coverage is still targeted, so very rare or highly complex legacy constructs may need manual intervention.

Q: How will you scale this project?

Add more transform rules, larger benchmarks, stronger ML ranking, and CI-backed quality gates.

From Scratch Learning Track

This chapter is written for beginners first. Each module starts with very simple meaning, then engineering depth, then practical action.

Module 1: Legacy Migration Pain Points

Simple view: legacy migration pain points explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, legacy migration pain points connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 2: Why Modernization Breaks At Runtime

Simple view: why modernization breaks at runtime explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, why modernization breaks at runtime connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 3: Scope Definition And Boundary Setting

Simple view: scope definition and boundary setting explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, scope definition and boundary setting connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 4: Why Deterministic Systems Build Trust

Simple view: why deterministic systems build trust explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, why deterministic systems build trust connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 5: How Diff-First Review Reduces Risk

Simple view: how diff-first review reduces risk explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, how diff-first review reduces risk connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 6: What Success Metrics Should Look Like

Simple view: what success metrics should look like explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, what success metrics should look like connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 7: How Students Can Deliver Practical Systems

Simple view: how students can deliver practical systems explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, how students can deliver practical systems connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 8: How Architecture Supports Future Scaling

Simple view: how architecture supports future scaling explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, how architecture supports future scaling connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 9: How To Communicate Limits Honestly

Simple view: how to communicate limits honestly explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, how to communicate limits honestly connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 10: How To Prioritize Roadmap Tasks

Simple view: how to prioritize roadmap tasks explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, how to prioritize roadmap tasks connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 11: How To Explain Hybrid ML + Rules

Simple view: how to explain hybrid ML + rules explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, how to explain hybrid ML + rules connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Module 12: How To Defend Project Value In Viva

Simple view: how to defend project value in viva explains one important part of this topic in plain language. If this part is weak, the whole workflow becomes harder to trust or harder to explain.

Deep view: in this project, how to defend project value in viva connects to concrete modules, endpoint contracts, and validation behavior. A good explanation should reference both logic and evidence.

Project evidence tie-in: backend LOC=1158, frontend LOC=2647, transforms=15, tests=10.

- Write this concept in your own 5-line explanation.
- Map this concept to one file and one function.
- Run one command or endpoint that demonstrates it.
- State one current limitation and one improvement idea.

Hands-On Practical Labs

1. Lab 1: Baseline Input Audit

Start from a legacy snippet and identify exactly what will break on modern Python.

- Collect one legacy input sample.
- Run /analyze and record issue ids.
- Classify each issue as syntax or semantic risk.
- Write one paragraph on expected modernization output.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

2. Lab 2: Analyze Output Interpretation

Understand how analysis payload maps to later planner and execution actions.

- Inspect probable_source_version and mode.
- Read legacy_issues list line by line.
- Map each issue id to a suggestion id.
- Explain why some items should become warning.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

3. Lab 3: Critic + Planner Trace

Trace how safety score and priority become selected_plans.

- Run analyze output through critique logic.
- Compute final score manually for two suggestions.
- Verify execution order in planner output.
- Compare manual and program outputs.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

4. Lab 4: Execution and Diff Review

Run modernization and inspect unified diff before applying changes.

- Execute /refactor on sample input.
- Inspect added and removed lines.
- Confirm each line links to a planned transform.
- Document one risky change for manual review.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

5. Lab 5: Validation Failure Drill

Learn fail-closed behavior by forcing a leftover-legacy construct.

- Create code where one legacy construct remains.
- Run validation and capture failure stage.
- Explain rollback behavior and user impact.
- Fix rule and rerun to green state.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

6. Lab 6: Run History and Reproducibility

Use run history to prove the pipeline is auditable and repeatable.

- Generate at least two runs with different inputs.
- Use /runs and /runs/{id} endpoints.
- Re-open one run in UI and inspect report tab.
- Write traceability summary notes.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

7. Lab 7: Topic-Specific Deep Task

Perform a deeper exercise related to idea concerns and document engineering tradeoffs.

- Choose one module strongly tied to this topic.
- Read code and summarize control flow.
- List strengths, risks, and missing checks.
- Propose one measurable improvement.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

8. Lab 8: Evaluation and Evidence Pack

Prepare evidence artifacts you can use in submission and viva.

- Run tests and benchmark script.
- Capture summary metrics and limitations.
- Align report claims with actual metrics.
- Store outputs in a clean docs folder.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

9. Lab 9: Explain to Beginner

Re-explain one complex part in simple words without losing correctness.

- Pick one algorithmic decision.
- Write a beginner explanation in 8 lines.
- Write an expert explanation in 8 lines.
- Ensure both versions match same truth.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

10. Lab 10: Full Demo Rehearsal

Do complete end-to-end dry run as if in final presentation.

- Start server and open UI.
- Run one safe case and one warning case.
- Show diff, report, and run history.
- Conclude with roadmap and limitations.

Expected output: produce notes with input, observed output, interpretation, and one risk/next-action line.

Troubleshooting Playbook

Issue	Likely Cause	How To Fix
Analysis returns no issues unexpectedly	Input pattern not covered by rules	Add/adjust analyzer rule and unit test.
Planner selects zero actions	Suggestions became NOOP or filtered by status	Inspect suggestion ids, critique map, and planner filters.
Validation fails after execution	Legacy leftovers or syntax problems remain	Inspect failure stage and patch transform logic.
Diff looks confusing to users	Output formatting or context lines are unclear	Improve diff rendering and report hints.
Run history missing expected data	RunStore payload fields not persisted	Verify run payload contract in save_run call.
ML shows unavailable	Adapter path missing or model disabled	Check env flags and adapter directory path.
UI button state feels wrong	State machine and action label out of sync	Trace state.primaryAction transitions.
Performance slows on large input	Heavy rendering or repeated processing	Paginate rendering and profile hot paths.
Benchmarks look too optimistic	Benchmark set too small or narrow	Expand dataset and stratify by pattern type.
Viva answers feel weak	No concrete evidence references	Quote tests, benchmark numbers, and code paths.
Topic-specific confusion in scope definition	Concepts missing state refs and flow	Add diagram plus file-level walkthrough.
Claims sound exaggerated	Scope and limitations not stated	Add explicit limitations and roadmap stages.

Glossary (Simple Words, Technical Meaning)

1. Legacy Pattern

An old syntax or API usage from earlier Python versions.

2. Modernization

Converting legacy code to current Python-compatible code.

3. Deterministic

Same input always gives same output.

4. Semantic Risk

Code may still run but behavior could change.

5. Validation Gate

Final checks that decide pass or rollback.

6. Diff-First Review

Inspect exact changes before applying to editor.

7. Analyzer

Stage that detects issues and risk indicators.

8. Suggester

Stage that maps detected issues to upgrades.

9. Critic

Stage that assigns safety scores and warnings.

10. Planner

Stage that chooses execution order.

11. Execution Core

Stage that applies selected transforms.

12. RunStore

Persistence layer for run history records.

13. LOCA (LOC)

Lines of code metric for rough project size.

14. Benchmark

Set of test cases used to measure pass rate.

15. Smoke Test

Small test set for quick health verification.

16. Rollback

Revert to original code when validation fails.

17. Adapter

LoRA fine-tuned parameter delta for base model.

18. Base Model

Original pretrained model used before fine-tuning.

19. Confidence

Estimated reliability signal for a suggestion.

20. Traceability

Ability to explain what changed and why.

21. Problem Framing

How the project problem is defined and bounded.

22. Scope Boundary

What the current version supports and excludes.

23. Roadmap

Planned future increments with measurable outcomes.

24. Value Proposition

Main benefit the project offers to users.

25. Feasibility

Whether project goals match available resources.

Extended Viva and Interview Q&A

Q1: What is the core goal of this report?

To explain this topic from beginner level to implementation depth with real project evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q2: How should I present this in viva?

Start with simple explanation, then show one concrete code path, then show one metric and one limitation.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q3: How do I avoid vague answers?

Always tie statements to file paths, endpoints, tests, or measured outputs.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q4: What if evaluator asks about limitations?

Answer directly and show roadmap with near-term measurable upgrades.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q5: How do I show project credibility?

Use deterministic workflow, validation behavior, and test/benchmark evidence.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q6: Why use simple language in technical docs?

It improves clarity and comprehension without reducing technical correctness.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q7: How should I practice this topic?

Run one lab task, inspect code, and explain output in your own words.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q8: How does this topic connect to full system?

Each topic is one layer of the same end-to-end modernization pipeline.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q9: How is this problem relevant in real world?

Many teams still maintain old scripts and internal tools that fail on modern Python runtimes.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q10: Why not solve every Python version edge case now?

Breadth-first support without validation depth would reduce reliability for a student project.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Q11: What is your crisp one-line vision?

Safe, transparent, and practical modernization for legacy Python code.

How to answer strongly: begin simple, cite one file/path evidence, cite one metric, then mention one limitation.

Code Walkthrough Appendix (With Explanation)

This appendix connects explanation to real files. Each excerpt includes simple-language meaning, technical interpretation, and why it matters before showing code lines.

Excerpt 1: Core API Flow

File: /Users/rutwikbhondave/Desktop/Helix AI Project/main.py

Simple explanation: this block handles API orchestration and full modernization flow control. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=4, classes=2, endpoint decorators=2.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: read_landing, health_check, read_landing, health_check.
- Endpoint decorators in excerpt: app.get, app.get.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import difflib
0002: import json
0003: import logging
0004: import os
0005: from datetime import datetime
0006:
0007: import uvicorn
0008: from fastapi import FastAPI, HTTPException, Request
0009: from fastapi.responses import HTMLResponse, JSONResponse
0010: from fastapi.staticfiles import StaticFiles
0011: from fastapi.templating import Jinja2Templates
0012: from pydantic import BaseModel
0013:
0014: from agents.analyzer import AnalyzerAgent
0015: from agents.critic import CriticAgent
0016: from agents.planner import PlannerAgent
0017: from agents.suggester import SuggestionAgent
0018: from core.config import settings
0019: from core.execution import ExecutionCore
0020: from core.ml_reasoner import MLReasoner
0021: from core.report_generator import ReportGenerator
0022: from core.run_store import RunStore
0023: from core.validation import ValidationCore
0024:
0025:
0026: analyzer = AnalyzerAgent()
0027: suggester = SuggestionAgent()
0028: critic = CriticAgent()
0029: planner = PlannerAgent()
0030: executor = ExecutionCore()
0031: validator = ValidationCore()
0032: reporter = ReportGenerator()
0033: run_store = RunStore(settings.RUN_HISTORY_FILE)
```

```

0034: reasoner = MLReasoner(
0035:     enabled=settings.ML_MODEL_ENABLED,
0036:     adapter_path=os.path.join(settings.BASE_DIR, settings.ML_MODEL_ADAPTER_PATH)
0037:     if not os.path.isabs(settings.ML_MODEL_ADAPTER_PATH)
0038:     else settings.ML_MODEL_ADAPTER_PATH,
0039:     base_model=settings.ML_MODEL_BASE,
0040: )
0041:
0042: app = FastAPI(
0043:     title=settings.APP_NAME,
0044:     version=settings.VERSION,
0045:     docs_url=None,
0046:     redoc_url=None,
0047: )
0048:
0049: app.mount("/static", StaticFiles(directory=settings.STATIC_DIR), name="static")
0050: templates = Jinja2Templates(directory=settings.TEMPLATE_DIR)
0051:
0052:
0053: class CodeRequest(BaseModel):
0054:     code: str
0055:
0056:
0057: class ContactRequest(BaseModel):
0058:     name: str
0059:     email: str
0060:     project: str
0061:     message: str
0062:
0063:
0064: INFO_PAGES = {
0065:     "mission": {
0066:         "title": "Our Mission",
0067:         "content": "<p>Helix AI modernizes legacy Python code with an autonomous multi-agent work
0068:     },
0069:     "team": {
0070:         "title": "The Team",
0071:         "content": "<p>A platform prototype built around agentic planning, rule-based modernization
0072:     },
0073:     "newsletter": {
0074:         "title": "Helix AI Updates",
0075:         "content": "<p>Track new migration rules, supported legacy patterns, and autonomous valida
0076:     },
0077:     "careers": {
0078:         "title": "Join Us",
0079:         "content": "<p>We are focused on developer tooling, code migration, and safe autonomous ex
0080:     },
0081:     "contact": {
0082:         "title": "Contact Us",
0083:         "content": "<p>Questions about legacy Python modernization, agentic workflows, or enterpr
0084:     },
0085:     "refund-policy": {
0086:         "title": "Project Scope",
0087:         "content": "<p>Helix AI focuses on legacy Python modernization. The current prototype str
0088:     },
0089:     "faqs": {
0090:         "title": "Frequently Asked Questions",
0091:         "content": "<p><strong>Q: What does Helix AI do today?</strong><br>A: It analyzes legacy P
0092:     },
0093:     "stories": {
0094:         "title": "Use Cases",
0095:         "content": "<p>Helix AI is designed for teams modernizing older Python services, education
0096:     },
0097:     "status": {
0098:         "title": "System Status",
0099:         "content": "<p>Modernization pipeline online.</p><ul class='list-disc pl-5 mt-4 space-y-2
0100:     },
0101: }
0102:

```

```
0103:
0104: @app.get("/", response_class=HTMLResponse)
0105: async def read_landing(request: Request):
0106:     return templates.TemplateResponse("new.html", {"request": request})
0107:
0108:
0109: @app.get("/health")
0110: async def health_check():
0111:     return JSONResponse(
0112:         content={
0113:             "status": "ok",
0114:             "app": settings.APP_NAME,
0115:             "version": settings.VERSION,
0116:             "ml_reasoner": reasoner.explain_status(),
0117:         }
0118:     )
0119:
0120:
```

Excerpt 2: Analyzer Rules

File: /Users/rutwikbhondave/Desktop/Helix AI Project/agents/analyzer.py

Simple explanation: this block handles legacy pattern detection and risk mode assignment. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=0, classes=0, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Regex operations are present, so edge-case pattern handling should be tested carefully.
- Risk/warning language exists in this block, so it affects safety signaling.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import ast
0002: import re
0003: from typing import Any, Dict, List
0004:
0005: from agents.base_agent import BaseAgent
0006:
0007:
0008: LEGACY_RULES = [
0009:     {
0010:         "id": "backtick_repr",
0011:         "pattern": re.compile(r"`[^\\n]+`"),
0012:         "source_hint": "Python 1.x/2.x",
0013:         "message": "Backtick repr syntax detected.",
0014:         "replacement": "Replace `expr` with repr(expr).",
0015:         "risk": "MEDIUM",
0016:     },
0017:     {
0018:         "id": "apply_builtin",
0019:         "pattern": re.compile(r"\bapply\s*\("),
0020:         "source_hint": "Python 1.x/2.x",
0021:         "message": "Legacy apply(...) builtin detected.",
0022:         "replacement": "Rewrite apply(fn, args[, kwargs]) as fn(*args[, **kwargs]).",
0023:         "risk": "MEDIUM",
0024:     },
0025:     {
0026:         "id": "print_statement",
0027:         "pattern": re.compile(r"^\s*print\s+[(\n].*$", re.MULTILINE),
0028:         "source_hint": "Python 2.x",
0029:         "message": "Legacy print statement detected.",
0030:         "replacement": "Wrap values with print(...).",
0031:         "risk": "LOW",
0032:     },
0033:     {
0034:         "id": "xrange_usage",
0035:         "pattern": re.compile(r"\bxrange\s*\("),
0036:         "source_hint": "Python 2.x",
0037:         "message": "xrange is not available in modern Python.",
0038:         "replacement": "Replace xrange(...) with range(...).",
0039:         "risk": "LOW",
0040:     },
0041:     {
```

```

0042:         "id": "raw_input_usage",
0043:         "pattern": re.compile(r"\braw_input\s*\("),
0044:         "source_hint": "Python 2.x",
0045:         "message": "raw_input was renamed in Python 3.",
0046:         "replacement": "Replace raw_input(...) with input(...).",
0047:         "risk": "LOW",
0048:     },
0049:     {
0050:         "id": "except_comma",
0051:         "pattern": re.compile(r"\s*except\s+[^:\n]+,\s*[A-Za-z_]\w*\s*:", re.MULTILINE),
0052:         "source_hint": "Python 2.x",
0053:         "message": "Old exception binding syntax detected.",
0054:         "replacement": "Convert except X, e: to except X as e:.",
0055:         "risk": "LOW",
0056:     },
0057:     {
0058:         "id": "not_equal_operator",
0059:         "pattern": re.compile(r"<>"),
0060:         "source_hint": "Python 2.x",
0061:         "message": "Legacy not-equal operator detected.",
0062:         "replacement": "Replace <> with !=",
0063:         "risk": "LOW",
0064:     },
0065:     {
0066:         "id": "iteritems_usage",
0067:         "pattern": re.compile(r"\.iteritems\s*\("),
0068:         "source_hint": "Python 2.x",
0069:         "message": "iteritems returns a Python 2 iterator API.",
0070:         "replacement": "Replace iteritems() with items().",
0071:         "risk": "LOW",
0072:     },
0073:     {
0074:         "id": "iterkeys_usage",
0075:         "pattern": re.compile(r"\.iterkeys\s*\("),
0076:         "source_hint": "Python 2.x",
0077:         "message": "iterkeys returns a Python 2 iterator API.",
0078:         "replacement": "Replace iterkeys() with keys().",
0079:         "risk": "LOW",
0080:     },
0081:     {
0082:         "id": "itervalues_usage",
0083:         "pattern": re.compile(r"\.itervalues\s*\("),
0084:         "source_hint": "Python 2.x",
0085:         "message": "itervalues returns a Python 2 iterator API.",
0086:         "replacement": "Replace itervalues() with values().",
0087:         "risk": "LOW",
0088:     },
0089:     {
0090:         "id": "has_key_usage",
0091:         "pattern": re.compile(r"\.has_key\s*\("),
0092:         "source_hint": "Python 2.x",
0093:         "message": "dict.has_key was removed.",
0094:         "replacement": "Rewrite d.has_key(x) as x in d.",
0095:         "risk": "MEDIUM",
0096:     },
0097:     {
0098:         "id": "unicode_type",
0099:         "pattern": re.compile(r"\bunicode\b"),
0100:         "source_hint": "Python 2.x",
0101:         "message": "unicode type detected.",
0102:         "replacement": "Replace unicode with str where compatible.",
0103:         "risk": "MEDIUM",
0104:     },
0105:     {
0106:         "id": "basestring_type",
0107:         "pattern": re.compile(r"\bbasestring\b"),
0108:         "source_hint": "Python 2.x",
0109:         "message": "basestring type detected.",
0110:         "replacement": "Replace basestring with str.",

```

```

0111:         "risk": "MEDIUM",
0112:     },
0113:     {
0114:         "id": "long_type",
0115:         "pattern": re.compile(r"\b(?:long\b)",
0116:         "source_hint": "Python 2.x",
0117:         "message": "long type detected.",
0118:         "replacement": "Replace long with int.",
0119:         "risk": "LOW",
0120:     },
0121:     {
0122:         "id": "exec_statement",
0123:         "pattern": re.compile(r"^\s*exec\s+[(\n].*$", re.MULTILINE),
0124:         "source_hint": "Python 1.x/2.x",
0125:         "message": "Legacy exec statement detected.",
0126:         "replacement": "Convert exec statement syntax to exec(...).",
0127:         "risk": "HIGH",
0128:     },
0129: ]
0130:
0131:
0132: HIGH_RISK_PATTERNS = [
0133:     {
0134:         "id": "division_semantics",
0135:         "pattern": re.compile(r"(?<!/)/(?!/)",
0136:         "message": "Division semantics may change between Python 2 and Python 3.",
0137:         "risk": "HIGH",
0138:     },
0139:     {
0140:         "id": "bytes_text_boundary",

```

Excerpt 3: Validation Gate

File: /Users/rutwikbhondave/Desktop/Helix AI Project/core/validation.py

Simple explanation: this block handles validation gate and rollback decision logic. It is an important piece in the from-input to validated-output workflow.

Technical explanation:

- Detected in this excerpt: functions=4, classes=1, endpoint decorators=0.
- This code should be read as part of a contract chain: what it receives, what it returns, and how the next stage depends on it.
- Key function names to read first: `validate`, `_check_lint`, `_verify_behavior`, `_get_function_names`.
- Regex operations are present, so edge-case pattern handling should be tested carefully.
- AST usage is present, indicating structured syntax analysis/validation behavior.

Why this matters:

- If this module fails, pipeline reliability drops immediately.
- Changes in this block should always be paired with targeted tests.
- When presenting in viva, mention one metric and one limitation tied to this code.

Code excerpt:

```
0001: import ast
0002: import re
0003: from typing import Dict, Any, List
0004:
0005:
0006: LEGACY_REMAINDER_PATTERNS = {
0007:     "backtick_repr": re.compile(r"`[^\\n]+`"),
0008:     "apply_builtin": re.compile(r"\bapply\s*\("),
0009:     "xrange": re.compile(r"\bxrange\s*\("),
0010:     "raw_input": re.compile(r"\braw_input\s*\("),
0011:     "iteritems": re.compile(r"\.iteritems\s*\("),
0012:     "iterkeys": re.compile(r"\.iterkeys\s*\("),
0013:     "itervalues": re.compile(r"\.itervalues\s*\("),
0014:     "has_key": re.compile(r"\.has_key\s*\("),
0015:     "not_equal": re.compile(r"<>"),
0016:     "except_comma": re.compile(r"except\s+([^\n:]+\s*,\s*([A-Za-z_]\w*)\s*:"),
0017:     "exec_statement": re.compile(r"^\s*exec\s+^[^\n].*$", re.MULTILINE),
0018: }
0019:
0020:
0021: class ValidationCore:
0022:     def validate(self, original_code: str, new_code: str) -> Dict[str, Any]:
0023:         warnings: List[str] = []
0024:
0025:         try:
0026:             ast.parse(new_code)
0027:         except SyntaxError as exc:
0028:             return {
0029:                 "success": False,
0030:                 "stage": "SYNTAX",
0031:                 "error": f"Syntax Error: {exc.msg} at line {exc.lineno}",
0032:                 "warnings": warnings,
0033:             }
0034:
0035:         lint_result = self._check_lint(new_code)
0036:         warnings.extend(lint_result.get("warnings", []))
0037:         if not lint_result["success"]:
0038:             return {
0039:                 "success": False,
```

```

0040:         "stage": "LINT",
0041:         "error": lint_result["error"],
0042:         "warnings": warnings,
0043:     }
0044:
0045:     behavior_result = self._verify_behavior(original_code, new_code)
0046:     warnings.extend(behavior_result.get("warnings", []))
0047:     if not behavior_result["success"]:
0048:         return {
0049:             "success": False,
0050:             "stage": "BEHAVIOR",
0051:             "error": behavior_result["error"],
0052:             "warnings": warnings,
0053:         }
0054:
0055:     return {
0056:         "success": True,
0057:         "stage": "COMPLETE",
0058:         "message": "Modernization validation passed.",
0059:         "warnings": warnings,
0060:     }
0061:
0062: def _check_lint(self, code: str) -> Dict[str, Any]:
0063:     warnings: List[str] = []
0064:     leftovers = [name for name, pattern in LEGACY_REMAINDER_PATTERNS.items() if pattern.search(
0065:     if leftovers:
0066:         return {
0067:             "success": False,
0068:             "error": f"Legacy constructs still remain after migration: {'', '.join(leftovers)}",
0069:             "warnings": warnings,
0070:         }
0071:
0072:     if re.search(r"(?<!/)/(?!/)", code):
0073:         warnings.append("Division operators remain; review Python 2 to 3 semantics manually.")
0074:
0075:     return {"success": True, "warnings": warnings}
0076:
0077: def _verify_behavior(self, original: str, new: str) -> Dict[str, Any]:
0078:     warnings: List[str] = []
0079:     original_functions = self._get_function_names(original)
0080:     new_functions = self._get_function_names(new)
0081:
0082:     missing = original_functions - new_functions
0083:     if missing:
0084:         return {
0085:             "success": False,
0086:             "error": f"Original functions disappeared during migration: {sorted(missing)}",
0087:             "warnings": warnings,
0088:         }
0089:
0090:     if "unicode" in original or "basestring" in original:
0091:         warnings.append("String type migration applied; bytes/text compatibility should be re
0092:
0093:     return {"success": True, "warnings": warnings}
0094:
0095: def _get_function_names(self, code: str) -> set:
0096:     try:
0097:         tree = ast.parse(code)
0098:     except SyntaxError:
0099:         return set()
0100:     return {node.name for node in ast.walk(tree) if isinstance(node, ast.FunctionDef)}

```